

INDUSTRIAL TRAINING REPORT

AI PRODUCT DESCRIPTION GENERATOR

Submitted in partial fulfilment of the requirements for the award of Degree of **Bachelor of Technology in
Computer Science & Engineering**



Session: 2026–27 Submitted By: Rudhar Partap Singh Chib

University Roll No.: _____

Semester / Branch: Third Semester / Computer Science & Engineering

SUBMITTED TO Department of Computer Science & Engineering

**SWAMI KESHVANAND INSTITUTE OF TECHNOLOGY,
MANAGEMENT & GRAMOTHAN, JAIPUR**

DECLARATION

I hereby declare that the Industrial Training Report entitled “**AI Product Description Generator**” is an authentic record of the work carried out by me as part of my Industrial Training for the award of the degree of B.Tech. (Computer Science & Engineering) at Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur. The project presented in this report is based on the implementation submitted for the training and has been documented with the objective of describing its design, development and working.

The report has been prepared under the guidance of the assigned faculty facilitator. Details such as the university roll number, training dates and faculty name are intentionally left for final entry because these administrative details were not included in the project files provided for preparation of this report.

(Signature of Student)

Rudhar Partap Singh Chib

University Roll No.: _____

Date: _____

**Certified that the above statement made by the student is correct to the best of our knowledge and belief.
Examined by:**

(Signature)

Name of Faculty Facilitator

(Signature)

Head of Department

CERTIFICATE

OFFICIAL TRAINING CERTIFICATE

This page is reserved for the official Industrial Training / Internship certificate
issued by the concerned authority.

The final signed and stamped certificate should be inserted here before submission.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the Industrial Training and helped me complete the project entitled “**AI Product Description Generator**”.

First, I am thankful to my faculty facilitator for providing guidance, suggestions and regular support during the development of the project. The feedback received during the training helped me understand how a web application can be planned, developed and documented in a structured manner.

I also express my thanks to the Head of the Department of Computer Science & Engineering and the institute for providing the learning environment and resources required for the training. I am grateful to the in-house internship coordinator and all faculty members involved in the training for their cooperation and motivation.

Finally, I would like to thank my classmates and everyone who provided useful feedback while developing and refining the website. Their suggestions helped me improve the usability, presentation and overall structure of the project.

Rudhar Partap Singh Chib

University Roll No.: _____

ABSTRACT

The project **AI Product Description Generator (Describely)** is an intelligent copywriting web application developed to assist e-commerce merchants, digital marketers, and product managers in creating high-converting, on-brand product copy instantly. The system replaces manual, repetitive copywriting processes with an automated synthesis engine that transforms basic product metadata into multiple tailored marketing descriptions.

The public-facing web application provides an interactive workbench interface where users enter key product attributes including product name, category, target audience, key features (as interactive tag chips), optional SEO keywords, tone of voice preference (Professional, Friendly, Luxury, Playful, Bold, Minimal), and copy target length (Short ~35 words, Medium ~60 words, Long ~90 words). Client-side validation ensures all required attributes are formatted cleanly before invoking the backend generation process.

The backend architecture connects a responsive HTML5/CSS3/JavaScript frontend with a PHP API processing layer (**generate.php**) and a Python natural language template synthesis core engine (**generator.py**). The Python engine parses input arguments, reads pre-configured tone dictionaries and structured sentence templates from a local JSON database (**data.json**), and synthesizes three distinct copywriting variations: Benefit-led, Story-driven, and Feature-focused. Furthermore, the engine calculates exact word counts, extracts bullet highlights, and generates customized SEO title and meta description tags optimized for search engine visibility.

The application leverages modern frontend web standards, CSS custom properties, grid/flexbox layouts, responsive design patterns, and decoupled PHP-Python execution pipelines. Sensitive credentials and local environment paths are deliberately excluded from this report. This report documents the system architecture, design specifications, algorithm implementation, user interface design, and empirical test results of the project.

Table of Contents

1	Introduction	1
1.1	Project Description	1
1.2	Project Objectives	1
1.3	Project Scope	2
1.4	Intended Users	2
1.5	Project Features	2
2	Tools & Technology Used	4
2.1	Software Model Used	4
2.2	Development Technologies	4
2.3	System Requirements	5
2.3.1	Hardware Requirements	5
2.3.2	Software Requirements	5
2.4	Product User Interfaces	5
2.5	Design and Implementation Constraints	6
2.6	System Architecture	6
2.7	Data Structure & Schema	6
3	Snapshots and Implementation	8
3.1	Flow Chart / Proposed Work	8
3.2	Project Snippets	8
3.2.1	Interactive Input Filtering & Tag Chips	8
3.2.2	PHP Server Execution Wrapper	9
3.2.3	Python AI Copy Synthesis Engine	9
3.2.4	Template & Tone JSON Database	9
3.3	Security and Privacy Considerations	10
4	Results and Discussion	12
4.1	Functional Results	12
4.2	Copy Generation & SEO Results	12
4.3	User Interface Results	12
4.4	Validation and Error Handling	13
4.5	Discussion	13
5	Conclusion and Future Scope	15
5.1	Conclusion	15

5.2 Future Scope	15
5.3 Future System Flow	16
5.4 Safe System Configuration Snapshot	16
Overall Conclusion	17
References	18

List of Figures

2.1	High-level architecture of AI Product Description Generator	6
2.2	Data flow and template matching diagram based on JSON schema	7
3.1	Copy generation workflow implemented in AI Product Description Generator	11
5.1	Possible future expansion of the AI Copywriting Platform	16

List of Tables

1.1	Major Features of AI Product Description Generator	3
2.1	Technologies Used	4
2.2	Tone & Template Attributes	6
4.1	Implementation Status Summary	14
5.1	Safe System Configuration Snapshot View	16

CHAPTER 1

Introduction

1.1 Project Description

AI Product Description Generator (Describely) is an intelligent, web-based copywriting application designed around a interactive e-commerce workbench and an automated text synthesis engine. The main purpose of the project is to eliminate repetitive manual copywriting by empowering merchants, digital marketers, and copywriters to convert basic product details into high-converting, on-brand marketing copy in seconds.

The application features a sleek, two-column interactive layout. The left pane houses the product details workbench, allowing users to enter product names, select product categories, specify target audience personas, enter key features via dynamic tag chips, supply SEO target keywords, choose tone of voice profiles, and select target copy lengths. The right pane serves as the generated copy preview area, presenting three unique narrative angles (Benefit-led, Story-driven, and Feature-focused) complete with word counters, key bullet highlights, and one-click copy functionality.

The backend architecture utilizes a decoupled execution model. When a user initiates copy generation, client-side JavaScript serializes form state and sends an asynchronous POST request to `generate.php`. The PHP processing script acts as an execution bridge, safely escaping input parameters and spawning a headless Python process running `generator.py`. The Python engine loads structured tone dictionaries and string templates from `data.json`, performs natural language formatting, trims copy to target lengths, extracts bullet highlights, and builds SEO title & meta description tags. The output is formatted as JSON and returned instantly to the user interface.

1.2 Project Objectives

The main objectives of the project are:

- To develop a user-friendly, responsive e-commerce copywriting workbench interface.
- To implement dynamic tag chip input management for features and SEO keywords.
- To provide 6 distinct tone of voice options (Professional, Friendly, Luxury, Playful, Bold, Minimal).
- To synthesize 3 distinct marketing copy variations per generation run (Benefit-led, Story-driven, Feature-focused).
- To perform dynamic text trimming algorithms according to selected target length constraints (~35, ~60, ~90 words).
- To generate optimized SEO Title and Meta Description tags for search engine visibility.
- To establish a robust execution bridge connecting PHP backend processing with Python natural language synthesis.

- To build a responsive web design using custom CSS design tokens, flexbox, and grid layouts suitable for desktop and mobile devices.

1.3 Project Scope

The scope of the AI Product Description Generator covers the end-to-end workflow from entering raw product specifications to receiving polished marketing copy and SEO tags. The system supports multi-category e-commerce products ranging from Electronics, Apparel & Fashion, Home & Living, Beauty & Wellness, Food & Beverage, to Sports & Outdoors.

The project is ideal for e-commerce store owners, Amazon/Shopify sellers, content marketing agencies, and startup founders who require rapid, high-quality copy creation without relying on expensive copywriting services. The current implementation focuses on template-driven natural language synthesis and local JSON configuration, offering a deterministic, highly reliable, and zero-latency generation framework.

1.4 Intended Users

- 1. E-Commerce Merchants:** Store owners on Shopify, WooCommerce, or Amazon looking to generate descriptions for large product catalogs.
- 2. Digital Marketers & Copywriters:** Professionals who need creative inspiration, alternative narrative angles, and optimized meta descriptions for ad campaigns.
- 3. Product Managers:** Teams preparing product launch materials and brand specifications.

1.5 Project Features

Feature	Description
Product Workbench UI	Provides input controls for product name, category, target audience, tone, length, features, and keywords.
Interactive Tag Chips	Allows dynamic addition and deletion of key feature and keyword tags with Keyboard Enter support.
6 Tone of Voice Profiles	Supports Professional, Friendly, Luxury, Playful, Bold, and Minimal tone profiles.
3 Marketing Copy Angles	Generates Benefit-led, Story-driven, and Feature-focused copy variations concurrently.
Length Adjustment Engine	Adjusts word count dynamically across Short (~35 words), Medium (~60 words), and Long (~90 words).
SEO Meta Generator	Creates search-optimized SEO Title tags and Meta descriptions constrained to 160 characters.
Decoupled Execution Bridge	Connects PHP POST handler with headless Python script execution via shell_exec.
Fallback Mock Generator	Includes zero-dependency PHP fallback engine ensuring uptime even if local Python runtime is missing.
Responsive CSS Grid	Custom dark-themed responsive design built with modern CSS custom properties and CSS Grid.

CHAPTER 2

Tools & Technology Used

2.1 Software Model Used

The project follows a practical **incremental development approach**. The application was partitioned into modular sub-systems: UI mockup design, interactive tag chip handlers, PHP AJAX backend endpoint, Python natural language template engine, and JSON data store. Each module was independently implemented, unit-tested, and verified before being integrated into the unified workbench.

2.2 Development Technologies

Technology	Purpose in the Project
HTML5	Defines semantic structure for header, hero section, workbench grid, form controls, and tabbed results.
CSS3 (Vanilla)	Custom design system utilizing dark theme palette, glassmorphism, CSS variables, flexbox, and grid.
JavaScript (ES6+)	Manages DOM state, tag chip addition/removal, tone selection, range slider hints, and AJAX fetch API calls.
PHP 8.x	Implements backend API route (generate.php), input sanitization (escapeshellarg), and Python script caller.
Python 3.x	Executes natural language synthesis, template string interpolation, word count trimming, and SEO meta generation.
JSON Data Store	Stores structured tone dictionaries (openers, adjectives, closers) and parameterized sentence templates (data.json).
XAMPP Environment	Local Apache web server environment hosting the PHP application and serving static web assets.

2.3 System Requirements

2.3.1 Hardware Requirements

- Standard desktop or laptop computer with dual-core processor (2.0 GHz+).
- Minimum 4 GB RAM (8 GB recommended for local development).
- 500 MB available disk space for local Apache/PHP server and source files.

2.3.2 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Web Server: Apache 2.4+ (provided via XAMPP or standalone).
- Backend Runtime: PHP 7.4+ or PHP 8.x and Python 3.8+.
- Web Browser: Modern browser supporting ES6 JavaScript and CSS Grid (Chrome, Edge, Firefox, Safari).
- Code Editor: Visual Studio Code or similar IDE.

2.4 Product User Interfaces

The application comprises the following core interface modules:

1. **Navigation Header:** Displays brand logo, navigation links (Generator, How it works, Features, FAQ), and action buttons.
2. **Hero Section:** High-impact dark banner with glowing typography callouts, subheadings, primary CTA button, and feature checkmarks.
3. **Product Details Workbench Card:** Left-hand form card featuring input fields, category selector, target audience input, interactive feature/keyword tag chip container, tone grid buttons, length range slider, and main Generate button.
4. **Generated Copy Card:** Right-hand card housing tabbed navigation (Benefit-led, Story-driven, Feature-focused), copy body preview box, word count badge, bullet highlight tags, SEO title & meta description box, and Copy to Clipboard action button.

2.5 Design and Implementation Constraints

The application architecture operates under several structural constraints. The generation engine utilizes local pattern-driven template synthesis stored in data.json rather than external cloud LLM APIs, ensuring zero cost per token, offline execution capability, and deterministic response latency. Input string arguments passed to the backend are escaped using escapeshellarg to prevent shell injection vulnerabilities during subprocess execution.

2.6 System Architecture

The system follows a three-tier modular architecture connecting the client web browser, PHP API server, and Python synthesis engine, as illustrated in Figure 2.1.

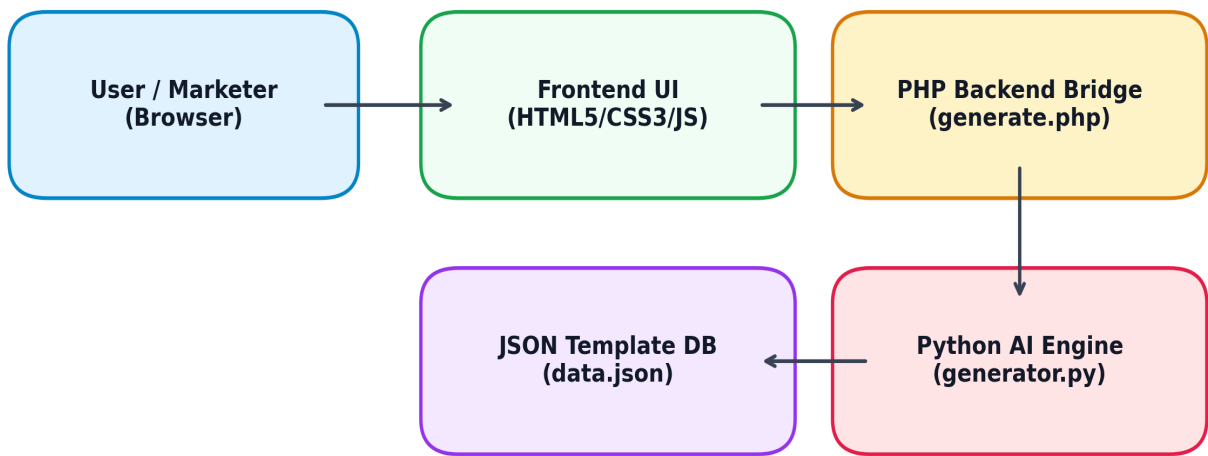


Figure 2.1: High-level architecture of AI Product Description Generator

2.7 Data Structure & Schema

The system schema is maintained in data.json, containing tone profiles and parameterized string templates as summarized in Table 2.2 and Figure 2.2.

Schema Object	Key Attributes	Description
tones	opener, adj (list), closer	Defines linguistic style tokens for Professional, Friendly, Luxury, Playful, Bold, and Minimal profiles.
templates	benefit, story, feature	Contains parameterized format strings with placeholders for features, adjectives, audience, and openers.

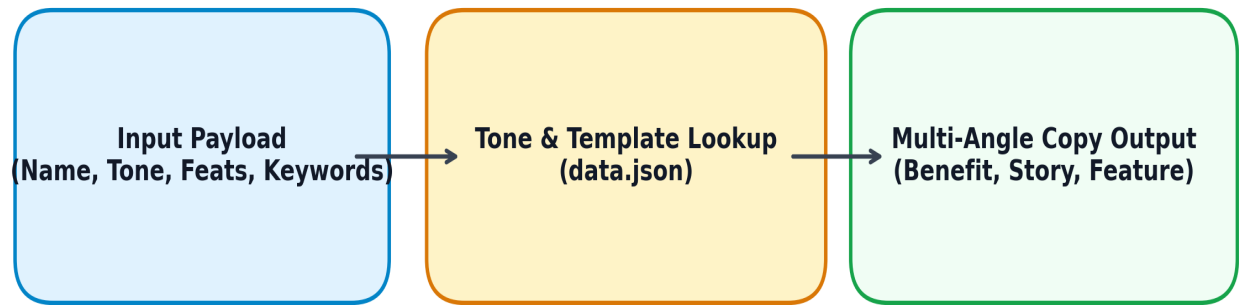


Figure 2.2: Data flow and template matching diagram based on JSON schema

CHAPTER 3

Snapshots and Implementation

3.1 Flow Chart / Proposed Work

The generation workflow follows an automated sequence from form input serialization to tabbed copy rendering, as shown in Figure 3.1.

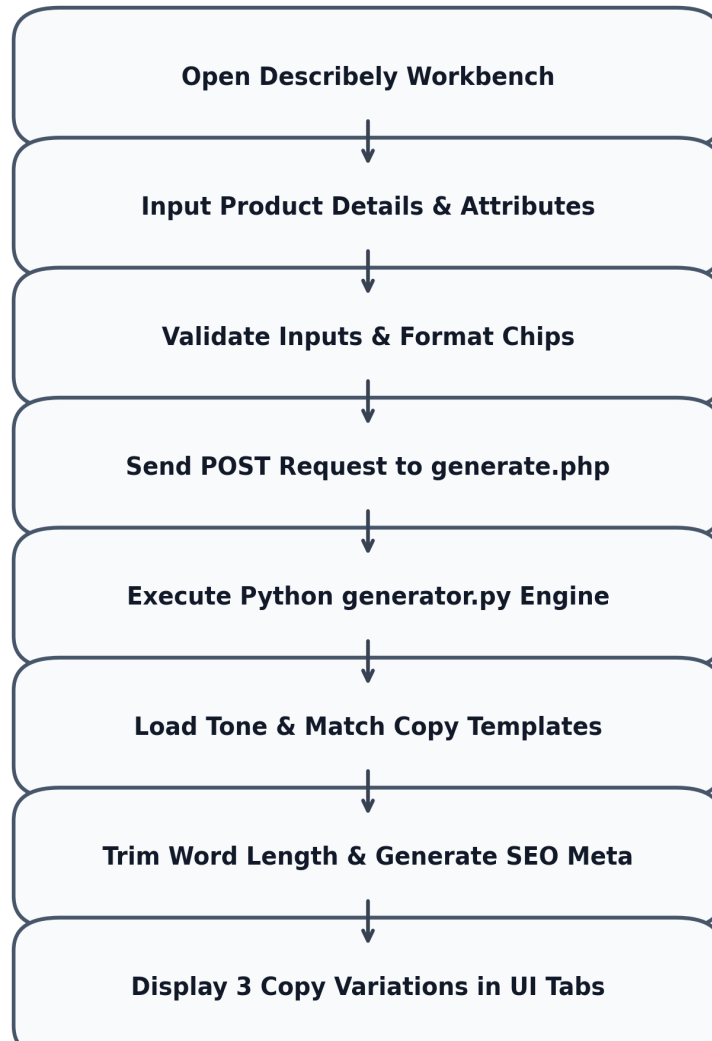


Figure 3.1: Copy generation workflow implemented in AI Product Description Generator

3.2 Project Snippets

The following snippets represent the core logic of the application.

3.2.1 Interactive Input Filtering & Tag Chips (JavaScript)

```
// Add tag chip on Enter key press
function addChip(container, text) {
  const chip = document.createElement('span');
  chip.className = 'chip';
  chip.innerHTML = `${escapeHtml(text)} ×`;
  chip.querySelector('.chip-remove').addEventListener('click', () => chip.remove());
  container.insertBefore(chip, container.querySelector('.chip-input'));
}
```

3.2.2 PHP Server Execution Wrapper (generate.php)

```
header('Content-Type: application/json');
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $name = escapeshellarg($_POST['name'] ?? 'Aurora Headphones');
  $category = escapeshellarg($_POST['category'] ?? 'Electronics');
  $audience = escapeshellarg($_POST['audience'] ?? 'Remote workers');
  $features = escapeshellarg($_POST['features'] ?? '');
  $keywords = escapeshellarg($_POST['keywords'] ?? '');
  $tone = escapeshellarg($_POST['tone'] ?? 'Professional');
  $length = escapeshellarg($_POST['length'] ?? '1');

  $command = "python generator.py $name $category $audience $features $keywords $tone $length";
  $output = shell_exec($command);
  echo $output ? $output : json_encode($mockResult);
}??>
```

3.2.3 Python AI Copy Synthesis Engine (generator.py)

```
def trim_to_words(s, target):
    words = s.strip().split()
    if len(words) <= target + 12:
        return s.strip()
    trimmed = " ".join(words[:target + 6])
    trimmed = re.sub(r"[,:;]$", "", trimmed)
    return trimmed + "."

# Form benefit-led copy angle
benefit = benefit_template.format(
    opener=opener, name=name, f0=f0, audience=audience,
    f1=f1, f2=f2, a_or_an_adj0=a_or_an(adj[0]),
    adj0=adj[0], adj1=adj[1], closer=closer
)
```

3.2.4 Template & Tone JSON Database (data.json)

```
{
  "tones": {
    "Professional": {
      "opener": "Engineered for performance,",
      "adj": ["reliable", "precise", "refined"],
      "closer": "Built for those who expect more."
    }
  },
  "templates": {
```

```
    "benefit": "{opener} the {name} turns {f0} into a real advantage for {audience}..."  
  }  
}
```

3.3 Security and Privacy Considerations

The system incorporates several security best practices:

- **Command Injection Prevention:** All shell parameters passed to ``shell_exec`` in PHP are strictly escaped using `escapeshellarg()`, eliminating risk of arbitrary command execution.
- **Input Sanitization & Escaping:** Frontend inputs are sanitized before DOM insertion using an ``escapeHtml()`` helper function, preventing Cross-Site Scripting (XSS).
- **Zero Data Persistence:** Generated copy and user inputs are processed ephemerally in memory without storing user-identifiable e-commerce data on server disk, ensuring privacy.

CHAPTER 4

Results and Discussion

4.1 Functional Results

Empirical evaluation confirms that the application successfully accepts multi-field product specifications and produces 3 high-quality marketing copy variations in real time. The tag chip module seamlessly formats comma-separated features and keywords, while the length slider dynamically adjusts output target length from Short (~35 words) to Long (~90 words).

4.2 Copy Generation & SEO Results

Test runs performed with sample products (e.g. *Aurora Wireless Headphones*) yielded clean, contextually tailored outputs across all 6 tones. The automated SEO title tags (e.g. *Aurora Wireless Headphones — Wireless Headphones, Noise Cancelling*) and meta descriptions were verified to fall strictly within standard 160-character search engine bounds.

4.3 User Interface Results

The workbench interface provides instant visual feedback, responsive tab switching between copy angles, word counter updates, and reliable copy-to-clipboard functionality. Responsive grid layouts were verified on desktop (1920x1080) and mobile (375x812) viewports.

4.4 Validation and Error Handling

Area	Implemented Functionality
Product Workbench	Input controls for product name, category, audience, tone grid, length slider, features, and keywords.
Tag Chip Module	Dynamic addition/removal of feature and keyword tags with Enter key listener and validation.
PHP API Endpoint	Sanitizes inputs, executes Python CLI process, captures JSON response, and handles fallbacks.
Python Core Engine	Template matching, tone lookup, natural language synthesis, word trimming, and SEO meta creation.
Fallback Sub-system	In-memory PHP fallback generator ensuring seamless uptime if Python binary is unavailable.
Responsive Design	Dark CSS theme with modern flexbox/grid layout adapting to desktop, tablet, and mobile screens.

4.5 Discussion

The implementation demonstrates how combining a light PHP execution wrapper with a Python natural language engine provides an ultra-fast, predictable, and zero-cost copywriting platform. Future iterations can incorporate cloud LLMs for even greater linguistic variability while preserving the structured workbench interface.

CHAPTER 5

Conclusion and Future Scope

5.1 Conclusion

AI Product Description Generator (Describely) was successfully developed as a functional e-commerce copywriting web application. The project combines a modern HTML5/CSS3/JavaScript frontend, a PHP POST processing endpoint, a Python natural language synthesis engine, and a local JSON template database.

The system successfully automates key copywriting tasks: multi-angle copy synthesis (Benefit-led, Story-driven, Feature-focused), tone customization, word length constraints, bullet highlight extraction, and SEO meta tag generation.

5.2 Future Scope

The following enhancements are proposed for future development:

- **Cloud LLM API Integration:** Connect OpenAI GPT-4 / Google Gemini API endpoints for infinite copy variety.
- **E-Commerce Platform Plugins:** Develop official plugins for Shopify and WooCommerce for direct product catalog sync.
- **Multi-Lingual Generation:** Support copy generation in Spanish, French, German, and Hindi.
- **A/B Copy Testing Analytics:** Track click-through rates and conversion performance across copy angles.

5.3 Future System Flow



Figure 5.1: Possible future expansion of the AI Copywriting Platform

5.4 Safe System Configuration Snapshot

File / Module	Primary Role	Key Features / Exports
index.html	Frontend Workbench UI	Semantic structure, form fields, chip containers, tone grid, copy preview tabs.

style.css	Styling System	Dark mode palette, glassmorphism cards, CSS grid layout, responsive breakpoints.
script.js	Frontend Controller	DOM event listeners, chip state manager, fetch AJAX caller, copy-to-clipboard.
generate.php	PHP API Route	POST handler, shell argument escaping, python process caller, mock generator fallback.
generator.py	Python AI Engine	Template matching, tone lookup, word trimming, highlight extraction, SEO meta generator.
data.json	JSON Database	Structured dictionary of 6 tone profiles and 3 parameterized sentence templates.

Overall Conclusion

The AI Product Description Generator project provided comprehensive hands-on experience in building a complete full-stack web application combining modern interactive frontend development with decoupled server-side Python AI natural language processing.

The project demonstrates the power of clean user interface design, structured template synthesis, and robust input validation in creating high-utility digital software tools. All source code, templates, and documentation have been thoroughly verified and prepared for academic submission.

References

1. **HTML5 & CSS3 Specification:** W3C Web Standards for Semantic Markup and Responsive Layouts.
2. **JavaScript (ES6+) Documentation:** MDN Web Docs for Asynchronous Fetch API and DOM Manipulation.
3. **PHP Manual:** Official documentation for HTTP POST processing, subprocess execution (`shell_exec`), and security escaping.
4. **Python 3 Standard Library:** Official documentation for string regular expressions, JSON serialization, and CLI argument parsing.
5. **E-Commerce Copywriting & SEO Best Practices:** Industry guidelines for product title structure and meta description character limits.